名称	修改日期	类型	大小
README	2026/1/28 19:46	md file	1 KB
TimeLine	2026/3/25 20:53	DOCX 文档	291 KB
大研课题规划建议	2026/3/25 20:53	WPS PDF 文档	1,396 KB

TimeLine.docx
文档

大研课题规划建议.pdf
PDF

这是数学建模课程主页
http://staff.ustc.edu.cn/~renjiec/mm2026/
作业素材有改变，目前数学建模老师只布置了第一次作业，有三个题目三选一，这是作业链接
https://github.com/TheVaticanCameos/USTC_MM_26/tree/master/hw_1，
附件图片中是我目前做出来的目录，还没有东西，根据之前我问你的大研课题规划建议，以及我做的Timeline，看看我现在需
要从哪儿开始，怎么做，要非常详细地，一步一步带着我做
我创建了专属的github仓库里：
https://github.com/YTYZ4/AI-for-research.git，目前还什么都没有

Thought for 35m 20s ›

我看完你的 Timeline、之前那份大研规划建议、课程作业仓库，以及你现在的 GitHub 之后，结论很明确：

你现在的起点不是继续想“大研整体怎么做”，而是立刻把 `hw_1` 的 `op_1`（接缝裁剪） 当成你大研的第一个主案例 A1 来做。
原因很简单：你的 Timeline 已经把首轮主案例定成 **A1 接缝裁剪 + A2 SVD 图像压缩**，而且 3 月这段时间本来就应该完成“收敛边界、补 Task Card、建 repo / runs / metrics / report、做第一条完整 run 记录”。你之前的大研规划建议也明确建议先做**单题闭环 demo**，再慢慢扩题库和原型。 TimeLine 大研课题规划建议

课程作业仓库也说明了：这学期会有 **4 次小作业 + 1 次大作业**，作业两周一次；目前第一次作业 `hw_1` 只需要在 3 个选项里**三选一**完成。 `hw_1` 的 README 里写得很清楚。
GitHub +1

这 3 个选项里， `op_1` 是**图像接缝裁剪**，要求实现 Seam Carving，并在报告里展示算法原理、长度/宽度缩放结果、与截断/伸缩的对比，还提供了 MATLAB / Python / C++ 三套

框架； `op_2` 是**城市地铁最短路**，要求建图 + 手写 Dijkstra； `op_3` 是 **karate club 网络的关键节点识别**，核心是度/接近/中介中心性，Optional 才到 PageRank 和 SIR。单看作业都能做，但**只有 `op_1` 和你已经写进 Timeline 的 A1 完全重合**，所以它最值。 GitHub +2

顺带说一句：你公开仓库现在**已经不是空仓库了**。我看到里面已经有 `data` / `metrics` / `outputs` / `problems` / `prompts` / `report` / `runs` / `solutions` / `task_cards` 这些目录，还有 `README.md` 和 `assignment1.pptx` / `assignment2.pptx`；README 里也已经写了 Task Card v0 的目标。所以你现在不是"从零开始"，而是"**骨架已经搭好，差第一份完整案例填进去**"。 GitHub +1

下面我不跟你讲空话，直接给你一套**从现在开始的执行顺序**。你就按这个顺序做，不要跳步。

第 1 步：先锁题，只做 `op_1`

先不要再比较三道题了，直接定：

- 课程作业： `hw_1/op_1`
- 大研主案例： `A1 = Seam Carving`
- 语言：**Python**
- 原因：你的仓库已经是 Python 结构， `op_1` 也给了 Python 框架，只需要补 `seam_carve_image(im, sz)`。 GitHub +1

你现在就在仓库里补这几个路径，不要改总目录结构：

```
AI-for-research/
  problems/
    hw1_op1_requirement.md
  task_cards/
    A1_seam_carving_taskcard_v1.md
  solutions/
    hw1_op1/
      reference_template/
      src/
      notes.md
  outputs/
    hw1_op1/
  runs/
    2026-03-25_run_001_requirement_parsing.md
    2026-03-25_run_002_template_setup.md
  metrics/
    hw1_op1_metrics.csv
  report/
    hw1_op1_report_outline.md
```

这一小步的完成标准：
你仓库里已经出现上面这些文件/目录，而且你脑子里不再有“到底选哪题”的摇摆。

第 2 步：把作业要求改写成你自己的“问题定义”

不要直接开始写代码。
先新建 `problems/hw1_op1_requirement.md` ，把课程要求**用你自己的话**重写一遍。

你只写 5 件事：

1. 题目目标是什么
2. 输入是什么
3. 输出是什么
4. 报告必须展示什么
5. 这次你**不做**什么

这里你要明确写出来：
这次作业的核心是实现 Seam Carving，并在报告中展示**算法原理、宽度/高度缩放结果、与截断/伸缩的对比、结果说明**；课程还给了 Python 模板，允许用自己的实现，但优先用模板最快。那些像 `Image Enlarging` 、 `Content Amplification` 之类扩展先不碰，除非基础版全做完。 GitHub

你可以直接按这个提纲写：

```
# hw1 op1 requirement
目标：实现内容感知的图像缩放，而不是普通 resize/crop。
输入：原图 im，目标尺寸 sz=(target_h, target_w)
输出：缩放后的图像，实验报告，代码
报告必须包含：
1. 算法基本原理
2. 缩小长度/宽度的结果
3. 与 crop / resize 的对比
4. 结果说明
本次暂不做：
1. 放大
2. content amplification
3. 多语言版本
```

完成标准：
你能不看 README，自己把题目给别人讲清楚。

第 3 步：把 Task Card 从 v0 升成 v1

你的 repo README 已经把 Task Card v0 的目标说清楚了：目标、输入输出、核心方法、可执行步骤、常见坑、评估指标。现在你要把这套东西真正落到 `A1` 上。 GitHub

新建 `task_cards/A1_seam_carving_taskcard_v1.md`，只写这 7 段：

1. 题目一句话目标
2. 输入 / 输出
3. 核心模型
4. 算法流程
5. 代码接口
6. 常见错误与自检
7. 评估指标与实验设计

这里你不用追求论文级推导，只要写到“你自己明天能照着实现”就够了。
核心模型只要写清三件事：

- 能量图 `E`
- 一条 seam 是从上到下的相邻路径
- 用动态规划找累计能量最小的 seam，再删掉它，重复直到达到目标宽度

算法流程也只写成最小实现顺序：

```
1. 计算能量图
2. 动态规划求累计最小代价
3. 回溯得到最优 seam
4. 删除一条 seam
5. 重复直到达到目标宽度
6. 高度缩放用转置复用
```

完成标准：
这张 Task Card 拿给“明天的你”，你能直接照它开工，不需要再去重新理解题目。

第 4 步：把官方模板先跑起来，别急着改算法

`op_1` 的 Python 文档给了环境依赖和启动方式：`numpy / matplotlib / scikit-image / scipy`，运行 `python seam_carving.py` 就会弹出交互窗口；文档也说 Python 版本和 MATLAB 版本接口等价，Python 里重点就是补 `seam_carve_image(im, sz)`。 GitHub

你先做这件事：

```
conda create -n mm26 python=3.12
conda activate mm26
```

```bash
pip install numpy matplotlib scikit-image scipy
```

上面这套命令就是 op1 Python 模板 README 里给的环境和运行方式。 GitHub

这一步有两个目的：

- 验证环境没问题
- 看清楚模板到底已经给了哪些辅助函数、哪些地方是 TODO

然后写 `runs/2026-03-25_run_002_template_setup.md` ，记录：

- 环境是否成功
- 模板能不能启动
- 你实际需要补哪些函数
- 你准备先走哪条最小路径

完成标准：
你已经看见 GUI 或至少确认脚本入口能跑，不再处于“我还没碰到代码”的状态。

第 5 步：先只做“宽度缩小”，做出第一个最小闭环

这一步最关键。
你不要一上来追求“宽高都能缩、还能放大、还能特别快”。
你只做最小版：

目标：让一张图成功删掉 1 条 seam，然后删掉 10 条 seam，然后删到目标宽度。

你按这个顺序写代码：

1. **先确认能量图**
   如果模板里已经有辅助能量函数，就复用；如果没有，就自己写一个最简单版本。
   先把能量图可视化保存出来，看是不是边缘高、平坦区域低。
2. **再做 DP 累计代价矩阵**
   你要得到每个像素作为 seam 终点时的最小累计代价。
3. **再做回溯**
   从最后一行最小值开始，一行一行往回找最优路径。
4. **再做 remove seam**
   对每一行删掉对应列，注意 RGB 三通道同步删。
5. **最后才循环多次**
   先删 1 条，成功后再删 10 条，再删到目标。

你这里每完成一个小点，都保存一个中间结果到 `outputs/hw1_op1/` ：

- `energy_map.png`

- `seam_overlay.png`
- `remove_1_seam.png`
- `remove_10_seams.png`

同时写 `runs/2026-03-25_run_003_first_working_version.md` ，记录你第一次跑通时：

- 输入图大小
- 输出图大小
- 是否报错
- 错在什么地方
- 你改了几轮

这一步最常见的错只有四种：

- DP 边界越界
- 回溯出来的 seam 不连续
- 删除后图像维度不对
- 删完一条 seam 没重算能量图

你每次调试都只盯这四件事，不要乱怀疑别的。

完成标准：
你已经得到一张“宽度明显变小，而且程序没崩”的结果图。

第 6 步：再补“高度缩小”，然后做对比实验

课程要求报告里不仅要有结果，还要有**对比实验**，比如和普通截断、普通伸缩比较。

GitHub

所以最省力的做法是：

- 宽度缩小：直接用你刚写好的 vertical seam
- 高度缩小：**转置图像**，复用同一套 vertical seam 逻辑，再转回来
- 对比方法：
  1. 普通 resize
  2. 直接 crop
  3. seam carving

至少做 2 张测试图：

- 一张主体明显的图（人物 / 建筑 / 动物）
- 一张横向信息很多的图（风景 / 城市场景）

你最后至少要保存这些图：

```
original.png
resize_linear.png
crop.png
seam_width.png
seam_height.png
compare_grid.png
```

然后建 `metrics/hw1_op1_metrics.csv` ，表头先别复杂，写这几个就够：

```
run_id,image_name,method,target_h,target_w,success,runtime_s,error_type,fix
```

这其实正好对应你的 Timeline 里已经计划好的指标：能跑率、报错类型、正确性/结果质量、耗时、修复轮次。 TimeLine

完成标准：
你手里已经有一张对比图和一张简单表格，报告素材基本齐了。

第 7 步：写报告，但先写“能交版”，别追求华丽

你在 `report/hw1_op1_report_outline.md` 里先搭 6 个标题：

1. 问题背景
2. 算法原理
3. 实现方法
4. 实验设计
5. 结果与分析
6. 局限与可能改进

每节写什么，我给你定死：

1. 问题背景

写为什么普通 resize / crop 不够好，为什么要内容感知缩放。

2. 算法原理

写能量图、最小能量 seam、动态规划思想。

3. 实现方法

写你代码里是怎样做能量、DP、回溯、删除的；高度缩小怎么通过转置复用。

4. 实验设计

写测试图、目标尺寸、对比方法。

5. 结果与分析

插入 `compare_grid.png` ，说明 seam carving 相比 resize / crop 保留了什么、破坏了什么。

6. 局限与可能改进

这里可以写两句就够：

- 当前只做了基础版
- 还没有做放大 / 速度优化 / 更复杂能量函数

完成标准：
你已经有一版“能交”的完整初稿，不再是零散笔记。

第 8 步：交作业之前，把它转成“大研 A1 的第一份资产”

这一步特别重要。
你不是只想把作业交掉，而是要把这次作业变成你大研的第一个样板案例。

你最后一定要把这 4 样东西留在仓库里：

1. `task_cards/A1_seam_carving_taskcard_v1.md`
2. `runs/` 里的 2–3 条 run 记录
3. `metrics/hw1_op1_metrics.csv`
4. `report/hw1_op1_report_outline.md` 和最终结果图

这样一来，你提交完第一次作业，实际上已经同时完成了 Timeline 里的四件事：

- 定了第一个主案例 A1
- 补了第一张 Task Card
- 有了第一条 run 记录
- 有了第一版 metrics 表

这和你 Timeline 里 P0 / W1-W3 的目标是完全一致的。 TimeLine

如果卡住，怎么判断要不要切换备选方案

我还是给你一个**保底判断线**：

- 如果你在 **第 4 步** 连模板都跑不起来，先只解决环境，不碰算法。
- 如果你在 **第 5 步** 花了很久，仍然连“删 1 条 seam”都做不出来，那就不要硬耗太久。

这时的保底方案是切到 `op_2` 。因为 `op_2` 的 Python 框架只需要补 `Graph` 、 `build_graph` 、 `dijkstra` 和接口调用，GUI 现成，验证路径也直接。它是一个很稳的保作业选项。 GitHub

但在你当前这条大研路线下，**只有在 op1 明显卡死**的时候才切。正常情况下不要换题。

你接下来这 48 小时就这样安排

今晚

完成第 1 ~ 4 步。
目标不是写出完美代码，而是把：

- 选题锁死
- requirement 写完
- Task Card v1 写完
- 模板跑起来

明天白天

完成第 5 ~ 6 步。
目标是：

- 宽度缩小先跑通
- 高度缩小补上
- 至少做出一组 seam / resize / crop 的对比图

明天晚上

完成第 7 ~ 8 步。
目标是：

- 报告初稿成型
- metrics 表成型
- run 记录成型
- 一次 commit 留档

最后一句最关键

你现在**不要**再把注意力放在“整个大研要做成什么样”。
你现在唯一该做的是：

把 `hw_1/op_1` 做成你大研的 A1 起始样板。

这一步做完，你后面所有东西都会顺：

- A1 Task Card 有了
- Prompt 模板有了落脚点
- metrics 表有了第一行

- `runs/` 有了第一条记录
- 给导师汇报时也终于不是“方向”，而是“产出物”

你先做完这 3 个文件再发我：

- `problems/hw1_op1_requirement.md`
- `task_cards/A1_seam_carving_taskcard_v1.md`
- `runs/2026-03-25_run_002_template_setup.md`

我就按你实际写出来的内容，继续带你做下一步。

来源

来源

第 8 步已完成，作业产物已经转成“大研 A1 第一份可复用资产”。

本次完成的封存动作

资产核验完成，4 类核心资产都在且可用。
Task Card: A1_seam_carving_taskcard_v1.md
Run 记录（已超过 2-3 条要求）: 2026-03-25_run_002_template_setup.md, 2026-03-25_run_003_first_working_version.md, 2026-03-25_run_004_height_and_comparison.md
Metrics 表: hw1_op1_metrics.csv
报告与结果图: hw1_op1_report_outline.md, compare_grid.png, compare_grid.png
新增了 Step 8 资产化记录。
2026-03-25_run_005_a1_asset_packaging.md
把“何时切换 op_2”的保底判据固化到项目笔记，避免后续摇摆。
notes.md
这条线现在和 Timeline 的目标对齐

已固定主案例 A1（Seam Carving）。
已有第一张 Task Card。
已有连续 run 记录链。
已有第一版 metrics 表。
结论
现在不是“交完一次作业就结束”，而是已经拿到一个可复用

的 A1 样板资产包。正常情况下继续沿 op_1 推进，不需要切题。
目前的成果已经推到了github仓库：
https://github.com/YTYZ4/AI-for-research.git

Thought for 35m 8s

很好，这一步算真正过线了。我刚核对了你公开仓库：A1 的 `task card v1` 、 `run 005` 、 `hw1_op1_report_outline.md` 、 `hw1_op1_metrics.csv` 都已经能直接读到；当前 A1 的 metrics 已经固定了两张测试图 `bing1.png` / `original.png` ，并记录了 `resize_linear` / `crop` / `seam_width` / `seam_height` 四种方法的结果和运行时间，所以“作业级闭环”已经成立。 GitHub +3

但我也看到一个很关键的短板：仓库首页 `README.md` 还只是 Task Card v0 的简短说明，没有把项目目标、A1 当前状态、目录索引和复现入口写出来。换句话说，资产已经有了，**入口层还没包装好。** GitHub +1

所以你现在的目标不再是“继续把 seam carving 做得更花”，而是把 A1 从**作业答案**升级成**研究对照平台**。报告里已经明确了当前版本只做 shrink，没有 enlarge，纯 Python 耗时偏长，能量函数也是基础版；这说明它已经够做研究基线，但现在还不值得你把主要时间花在放大和性能优化上。 GitHub +1

现在开始，按这个顺序做

1. 先把仓库入口整理好

今天第一件事就是重写顶层 `README.md` 。

你不要写成长篇介绍，只要 6 段：

```
# AI-for-research
一句话目标：用 A1/A2 主案例做“过程化指导 + 结构化检查 + 纠错评估”的研究仓库。

## Current status
- A1 Seam Carving：已完成作业闭环，已有 task card / runs / metrics / report / outpu
- A2：待启动
- 当前阶段：把 A1 从作业资产升级为研究对照基线

## Quick links
- task_cards/A1_seam_carving_taskcard_v1.md
- runs/2026-03-25_run_005_a1_asset_packaging.md
- report/hw1_op1_report_outline.md
```

```
- metrics/hw1_op1_metrics.csv

## Repo map
task_cards / prompts / runs / metrics / report / solutions / outputs

## Reproduce A1
环境、入口脚本、数据路径、输出位置

## Next milestones
1. A1 baseline vs CoE
2. A1 failure cases
3. A2 bootstrap
```

这一步看起来像“文档活”，其实非常关键。因为你后面给导师、给同学、甚至给未来的自己看仓库，第一眼先看的不是 `solutions/` ，而是首页。

完成标准只有一个：
别人点开仓库首页，**30 秒内知道这是什么、做到哪了、下一步是什么。**

---

2. 把指标分成两层，不要混在一个表里

你现有的 `metrics/hw1_op1_metrics.csv` 很有价值，但它本质上是**作业/算法层指标**：它记录的是 `resize_linear` / `crop` / `seam_width` / `seam_height` 这些图像处理方法的结果与运行时间。也就是说，它衡量的是“算法效果”，不是“AI 指导方法效果”。 GitHub

所以现在不要改坏它。
你应该：

- 保留 `metrics/hw1_op1_metrics.csv` 作为 **assignment-level metrics**
- 新建 `metrics/a1_guidance_eval_v0.csv` 作为 **research-level metrics**

新表头直接用这个：

```csv
run_id,case_id,mode,prompt_file,task_card_version,artifact_complete,runnable,corr
```

字段解释你按这个口径定：

- `mode`：`direct_answer` / `plain_guidance` / `coe_guided`
- `artifact_complete`：有没有按要求给出核心函数、输出图、报告要点
- `runnable`：能不能真正跑通
- `correct`：是否满足 A1 的基本功能

- `self_check`：有没有主动检查 seam 连续性、维度、能量重算等
- `error_type`：归到你的错误分类
- `fix_rounds`：修几轮才工作
- `time_to_first_working_min`：从开始到第一次跑通花了多久

这一步是你后面中期能不能讲“研究结果”的关键。
因为你要比较的不是单纯的 seam carving 算法，而是：

- 直接给答案
- 普通辅助
- 多角色/结构化指导

到底哪种方式更稳、更容易纠错。

---

3. 固定 A1 的评测协议，后面所有 run 都别乱改

你现在 A1 已经有一套可重复的实验素材了：报告里用的是 `bing1.png` 和 `original.png` 两张图，run 004 里还注明了为了控制运行时间，输入会先压到 `max_side=420` 。这套口径一旦固定，后面所有 baseline 和 CoE run 都沿用，不要今天换图、明天换尺寸，不然数据没法比。 GitHub +2

你把这三条写成 `report/a1_eval_protocol_v0.md` ：

```
# A1 evaluation protocol v0
固定测试图:
1. bing1.png
2. original.png

固定预处理:
- max_side = 420

固定目标任务:
1. width shrink
2. height shrink

固定作业级对照:
1. resize_linear
2. crop
3. seam_width
4. seam_height

固定研究级对照:
1. direct_answer
```

```
2. plain_guidance
3. coe_guided
```

这样你后面每多跑一轮，都能直接落表，不需要重新解释“这次为什么不一样”。

---

4. 先做第一次研究级 baseline：不是再写代码，而是比较“怎么被指导”

这一步最重要。

你现在已经有一份能工作的 **gold A1**。
从现在开始，不要拿这份金标准反复覆盖。把它冻结下来，例如在本地打一个 tag，或者至少明确写：

- `solutions/hw1_op1/gold/` ：你当前已经跑通的版本
- `solutions/hw1_op1/generated/` ：后面不同提示方式生成的版本
- `solutions/hw1_op1/failure_cases/` ：故意构造的错误样例

然后连续跑 3 个模式：

模式 A：direct_answer
只给题目要求，不给 Task Card，不给错误清单，让模型直接出方案/代码。

保存：

- `prompts/a1/baseline_direct_answer_v0.md`
- `runs/2026-03-26_run_006_a1_direct_answer_baseline.md`

模式 B：plain_guidance
给题目要求 + Task Card v1，但不使用多角色分工。

保存：

- `prompts/a1/baseline_plain_guidance_v0.md`
- `runs/2026-03-26_run_007_a1_plain_guidance_baseline.md`

模式 C：coe_guided
给题目要求 + Task Card v1 + 多角色输出格式。

保存：

- `prompts/a1/coe_multi_role_v0.md`
- `runs/2026-03-27_run_008_a1_coe_guided.md`

每跑完一个模式，你都做同样 4 件事：

1. 把生成代码单独存到 `generated/run_xxx/`

2. 用固定协议去跑
3. 把结果填进 `a1_guidance_eval_v0.csv`
4. 写 run 记录：哪一步出错、怎么修、最后能不能跑通

这样你第一次真正有了“baseline vs your method”的研究对照。

---

5. 多角色 CoE 的输出格式，直接这样定

你不要让多角色流程太虚。
A1 的 Task Card 已经给了非常明确的接口拆分和常见错误：主接口 `seam_carve_image(im, sz)`，最小函数拆分是 `compute_energy / find_vertical_seam / remove_vertical_seam / seam_carve_width / seam_carve_height`，而且已经列出 5 类高频错误：DP 边界、seam 不连续、删 seam 后维度不对、转置复用出错、dtype/像素范围异常。Reviewer 的检查表就直接从这里抄，不要另起炉灶。 GitHub

你的 `coe_multi_role_v0.md` 直接规定 5 个角色，每个角色必须输出固定标题：

```
## Role 1 Reader
- 题目重述
- 输入/输出
- 验收条件

## Role 2 Planner
- 算法步骤
- 函数拆分
- 边界与风险点

## Role 3 Coder
- 代码骨架
- 关键实现
- 需要自检的断言

## Role 4 Reviewer
- 对照 Task Card 的 5 类常见错误逐项检查
- 列出最可能失败点
- 给修复建议

## Role 5 Experimenter
- 固定测试图
- 固定目标尺寸
- 对比方法
- 记录项与结果总结
```

你后面所有 A1 的 CoE run 都必须遵守这个输出结构。
这样到 A2 的时候才能真正复用。

---

6. 把 A1 的“错误样例包”做出来，这是后面最值钱的教学资产

Task Card v1 里那 5 类错误，其实已经是你最好的 failure case 来源。 GitHub

你先只做人手可控的 3 个：

```
solutions/hw1_op1/failure_cases/
  bug_01_dp_boundary/
  bug_02_no_energy_recompute/
  bug_03_height_transpose/
```

每个错误样例目录里放 4 个东西：

- `buggy_code.py`
- `symptom.md`
- `diagnosis.md`
- `fix.patch` 或 `fixed_code.py`

你写 `symptom.md` 的格式统一成：

```
# symptom
现象：
- 报什么错 / 输出什么异常结果

触发条件：
- 哪张图
- 哪个 target size

归类：
- 建模错误 / 代码错误 / 求解异常 / 结果解释不足
```

这一步非常重要。因为你后面做“AI 用来教学纠错”时，最值钱的不是金标准代码，而是**有代表性的错法 + 对应反馈规则。**

---

7. 这周最后才开 A2，而且只开壳，不急着实现

你现在千万别一股脑跳去写 A2 的完整算法。
正确顺序是：

1. A1 baseline 跑完

2. A1 CoE 跑完
3. A1 failure cases 出来
4. 再开 A2

A2 这周只做 3 个文件就够：

```
problems/a2_requirement.md
task_cards/A2_taskcard_v0.md
runs/2026-03-27_run_009_a2_bootstrap.md
```

A2 的目标不是这周写完，而是先把第二主案例的**问题定义、接口、评测口径**立起来。
这样你的研究就从“只有一个作业样板”变成“有第二案例正在接上”。

你这周的完成标准

不要贪多。
这周只盯这几样：

```
README.md
report/a1_eval_protocol_v0.md
metrics/a1_guidance_eval_v0.csv
prompts/a1/baseline_direct_answer_v0.md
prompts/a1/baseline_plain_guidance_v0.md
prompts/a1/coe_multi_role_v0.md
runs/2026-03-26_run_006_a1_direct_answer_baseline.md
runs/2026-03-26_run_007_a1_plain_guidance_baseline.md
runs/2026-03-27_run_008_a1_coe_guided.md
solutions/hw1_op1/failure_cases/...
```

如果这批东西出来了，你的状态就会从：

“我做完了第一次作业”

变成：

“我已经有了 A1 的金标准、研究级 baseline、CoE 模板、错误样例包，A2 也准备启动。”

这才是大研真正往前走了一步。

你先别继续改 seam carving 代码本体。先去做 `README.md` 、 `a1_guidance_eval_v0.csv` 、 `run_006/007/008` 这三块。

来源

来源

目前的成果已经推到了github仓库：
https://github.com/YTYZ4/AI-for-research.git
本次完成内容

固定了 A1 的多角色 CoE 输出协议
已重写 coe_multi_role_v0.md
已强制 5 个角色固定标题：
Role 1 Reader
Role 2 Planner
Role 3 Coder
Role 4 Reviewer
Role 5 Experimenter
Reviewer 检查项已直接对齐 Task Card 的 5 类高频错误，不再另起规则
文档里已写明后续 A1 CoE run 必须遵守该结构
做出了 A1 错误样例包（3 个可控 failure cases）
根目录说明： README.md
已创建：
bug_01_dp_boundary
bug_02_no_energy_recompute
bug_03_height_transpose
每个目录都包含 4 个文件：
buggy_code.py
symptom.md
diagnosis.md
fixed_code.py
symptom.md 已按你给的统一模板书写（现象/触发条件/归类）
完成了 A2 本周只开壳（不做实现）
a2_requirement.md
A2_taskcard_v0.md
2026-03-27_run_009_a2_bootstrap.md
你前面要求的周核心资产保持可用
协议文件已在 a1_eval_protocol_v0.md
研究级指标表仍在 a1_guidance_eval_v0.csv
三个 prompt 文件齐全于 a1
run 006/007/008 文件已保留并可直接回填
当前已经达到的状态
不是“做完一次作业”，而是“已经有 A1 金标准、研究级

baseline 跑道、固定 CoE 模板、可教学错误样例包，且 A2 已启动壳”。这就是可持续研究管线的形态。

Thought for 19m 20s ›

这次我同意你的判断：**仓库已经不是“作业存档”，而是一个有研究管线雏形的项目。**现在顶层 `README` 已经把项目一句话目标、A1/A2 状态、A1 复现入口和 next milestones 写出来了；A1 的 CoE prompt 也已经固定成 5 个角色，并把 Reviewer 检查表直接对齐到 Task Card 的 5 类高频错误；A1 的评测协议也冻结了测试图、预处理和三种研究级对照；A2 的 requirement、task card 和 bootstrap run 也都已经落库。 GitHub +5

你做出的 3 个 failure cases 也很值钱，因为它们已经从“抽象错误类型”变成了最小复现材料：目录下有统一的 README，三个 bug 的 `symptom.md` 也都写了触发图、目标尺寸和归类，这已经足够支持后面的教学纠错实验了。 GitHub +3

现在真正的瓶颈只剩一个：研究级对照数据还没有跑出来。 你的 `a1_guidance_eval_v0.csv` 里，run 006/007/008 三行还是 `pending`；对应的三个 run 文档里，`status / first_error / fix_steps / runnable / correct` 也都还是待填。这说明结构已经齐了，但“baseline vs plain guidance vs CoE” 还没有变成可分析的数据。 GitHub +3

所以你接下来不要再扩目录，也不要继续加 A1 新功能。**下一阶段只做一件事：把 run 006/007/008 真正执行完，填满 metrics，产出第一份比较结论。**

你现在该怎么做

1. 先冻结评测记分规则

在真正跑 006/007/008 之前，先补一个很小但很关键的文件：

`metrics/a1_guidance_eval_codebook_v0.md`

把 `a1_guidance_eval_v0.csv` 里几个字段的取值先定死，不然后面你会越填越不一致。我建议你就这样定：

- `artifact_complete`： `0=缺核心产物`， `1=有代码但产物不全`， `2=代码+对比图+run记录齐`
- `runnable`： `0=不能跑`， `1=能跑`
- `correct`： `0=不满足A1`， `1=部分满足（只宽或只高）`， `2=宽/高/compare_grid都满足`
- `self_check`： `0=无`， `1=口头提到检查`， `2=有明确断言/验证步骤`
- `error_type`：固定成 `dp_boundary / seam_discontinuous / no_energy_recompute / height_transpose / dtype_range / other / none`

这一步做完以后， `a1_guidance_eval_v0.csv` 才真正能作为研究表来用。当前 CSV 只有表头和 3 条 planned run，占位已经有了，现在缺的是统一填表口径。 GitHub

2. 跑 run 006，但要按“实验”方式跑，不是随手聊天

`baseline_direct_answer_v0.md` 已经把 run 006 的输入边界定好了：只给题目要求、目标函数、输出要求和 shrink-only 限制，再按 A1 固定协议跑。 GitHub +1

你执行 run 006 时，顺序固定成这样：

1. 用同一个模型、同一个环境、同一个固定协议开始。
2. 把你发给模型的完整提示词原样保存到 `solutions/hw1_op1/generated/run_006_direct_answer/prompt_used.md`
3. 把模型原始回复完整保存到 `solutions/hw1_op1/generated/run_006_direct_answer/model_raw_response.md`
4. 从回复里抽取代码，放到 `solutions/hw1_op1/generated/run_006_direct_answer/src/`
5. 输出目录单独分开，不要覆盖 gold 版：`outputs/hw1_op1/run_006_direct_answer/`
6. 严格按固定协议跑两张图、两类 shrink、三种研究级模式口径中的 direct_answer 这一条。
7. 跑完立刻填：
   - `runs/2026-03-26_run_006_a1_direct_answer_baseline.md`
   - `metrics/a1_guidance_eval_v0.csv`

这里最重要的是**保留原始回复**。因为你的研究对象不是只有“代码能不能跑”，还包括“指导方式输出了什么结构、有没有主动自检、第一次失败点在哪”。run 006 的 run 文档已经预留了这些字段。 GitHub +1

3. 再跑 run 007，唯一变化是加 Task Card，不加多角色

`baseline_plain_guidance_v0.md` 已经把 plain guidance 的边界写清楚了：给作业要求 + `Task Card v1` ，但不做多角色拆分，仍然按固定协议评测。 GitHub +1

run 007 的执行方式和 006 完全一样，只改三件事：

- prompt 文件换成 `prompts/a1/baseline_plain_guidance_v0.md`
- 代码目录换成 `generated/run_007_plain_guidance/`
- 输出目录换成 `outputs/hw1_op1/run_007_plain_guidance/`

你要特别记录两点：

- 相比 006，它第一次出错时是不是更接近 Task Card 里的 5 类高频错误之一
- 它有没有出现“会说检查，但实际上没做检查”的情况

因为这一步的核心不是证明它一定更好，而是测它**是否比 direct_answer 更稳定、更少走弯路**。run 007 的目标和填表位置已经在仓库里预留好了。 GitHub +1

4. 最后跑 run 008，这是你现在最关键的一次

`coe_multi_role_v0.md` 已经把 5 个角色、固定标题、Reviewer 检查表和落地写法都锁死了，而且明确要求后续所有 A1 CoE run 都遵守这个结构，并把结果记录到 run 008 和 guidance eval CSV。 GitHub

你跑 run 008 时要盯这 4 个问题：

1. Reader 有没有把验收条件说清楚
2. Planner 有没有把函数拆分写完整
3. Coder 有没有写出自检断言
4. Reviewer 有没有真的按 5 类错误逐项检查，而不是空泛评价

只要其中两三项明显优于 006/007，你就已经有第一条很像“研究结论”的东西了：
固定多角色结构，不一定直接提高算法质量，但很可能提高首轮输出的可检查性与纠错效率。

这不是最后结论，但足够写进第一份比较 memo。run 008 的仓库骨架已经准备好了。 GitHub +2

5. 跑完三条以后，立刻写一个一页式比较说明

不要等很久。
你把三条都跑完以后，马上新建：

`report/a1_baseline_vs_coe_v0.md`

只写 4 段：

1. **实验设置**
   三个模式、固定测试图、固定协议、同一执行环境
2. **结果表**
   直接贴一张小表：`mode / runnable / correct / fix_rounds / time_to_first_working_min / error_type / self_check`
3. **观察**
   哪种模式首轮最稳，哪种模式修得最少，哪种模式最会“主动检查”
4. **结论 v0**
   先写临时结论，不要写大话

这一步很重要，因为现在你的仓库里已经有了 A1 协议、三个模式 prompt、三个待执行 run 和 research-level metrics 表；一旦把三条结果填进去，就能自然接出第一份比较报告。 GitHub +7

6. A2 现在继续按住，不实现

你这一步做得对。A2 的 task card 和 bootstrap run 都明确写了：本周只是建壳，不进入完整实现，要等 A1 的 baseline 跑完之后再启动 A2 的接口级实现。现在不要破坏这个顺序。 GitHub +1

我建议你这两天只盯这 5 个交付物

1. `metrics/a1_guidance_eval_codebook_v0.md`
2. 填完的 `runs/2026-03-26_run_006_a1_direct_answer_baseline.md`
3. 填完的 `runs/2026-03-26_run_007_a1_plain_guidance_baseline.md`
4. 填完的 `runs/2026-03-27_run_008_a1_coe_guided.md`
5. `report/a1_baseline_vs_coe_v0.md`

再补一个很小的润色动作：你三个 `symptom.md` 里目前还保留着模板式占位语句，比如“报什么错 / 输出什么异常结果”“哪张图”这类提示词，虽然下面已经跟了具体实例，但正式展示前最好把这些占位语句删掉，只留下真实症状、真实触发条件和最终归类。 GitHub +2

你现在已经过了“继续搭架子”的阶段。下一步不是再建文件夹，而是**把 006/007/008 跑成第一组可比较数据**。完成这一步，你的项目就会第一次从“研究设计”真正进入“研究结果”。

来源

目前的成果已经推到了github仓库：
https://github.com/YTYZ4/AI-for-research.git
已完成，你要的“从设计转到结果”这一步已经落地，
006/007/008 三条都跑成了可比较数据。

本次完成的核心交付

已执行并完成 run 006（direct_answer）
运行记录已填完: 2026-03-
26_run_006_a1_direct_answer_baseline.md
实验资产已保存:
prompt_used.md
model_raw_response.md
src
run_006_direct_answer
已执行并完成 run 007（plain_guidance）
运行记录已填完: 2026-03-
26_run_007_a1_plain_guidance_baseline.md
实验资产已保存:
prompt_used.md

model_raw_response.md
src
run_007_plain_guidance
已执行并完成 run 008（coe_guided，关键轮）
运行记录已填完: 2026-03-27_run_008_a1_coe_guided.md
实验资产已保存:
prompt_used.md
model_raw_response.md
src
run_008_coe_guided
guidance 研究表已从 pending 变为可分析数据
已更新: a1_guidance_eval_v0.csv
一页式比较说明已产出
已新建: a1_baseline_vs_coe_v0.md
你要求的 symptom 占位语句已清理完成
symptom.md
symptom.md
symptom.md
当前可直接引用的 v0 结论

三种模式都可跑通且满足 A1 基本正确性（correct=2）。
direct_answer 出现了首轮路径配置错误（fix_rounds=1）。
plain_guidance 与 coe_guided 在首轮可运行性更稳（fix_rounds=0）。
coe_guided 的输出结构与检查覆盖度最好（self_check=2，Reviewer 五类错误逐项检查到位），但本轮耗时最高。
A2 这一步我没有推进实现，保持了你要求的顺序。

Thought for 20m 10s

很好，这次算是真正拿到**第一份可讲、可复查、可继续扩展的研究结果**了。你仓库里现在已经同时具备：顶层入口 README、A1 的 guidance 评分 codebook、固定 5 角色的 CoE prompt、完成态的 run 006/007/008，以及一页式的 `A1 baseline vs CoE v0` 比较说明；A2 也还保持在 bootstrap-only，没有越界提前实现。 GitHub +4

你现在这条 v0 结论是站得住的：三种模式都跑通且达到 `correct=2`； `direct_answer` 首轮因为路径配置问题多修了一轮，而 `plain_guidance` 和 `coe_guided` 都是首轮可运行；CoE 的自检覆盖最好，明确把 Reader/Planner/Coder/Reviewer/Experimenter 五块都落

了下来，但 `time_to_first_working_min` 也是三者里最高。你的比较 memo 也已经把这个结论写清楚了。 GitHub +4

不过现在还有一个很重要的事实：**这还是 v0，而且本质上是每种模式各 1 次的首轮比较。**你的比较报告自己也写了，“下一轮应继续用同协议复跑，观察趋势是否稳定，而不是立刻扩大结论范围。”这句话非常对。 GitHub

我建议你下一步不要立刻去写 A2 算法，而是进入一个很明确的 Phase 2：**复跑验证 + 错误修复基准 + 再启动 A2。**

1. 先把顶层 README 更新一次。
   你现在 README 里的 `Current status` 和 `Next milestones` 还停留在“把 A1 从作业资产升级为研究对照基线”“A1 baseline vs CoE / A1 failure cases / A2 bootstrap”，但这些实际上都已经做完了，README 已经落后于仓库真实状态。下一版首页建议改成：
   `Current status: A1 v0 result completed; baseline/plain/CoE finished; failure cases ready; A2 still bootstrap-only.`
   `Next milestones: A1 replication / A1 bug-repair benchmark / A2 minimal implementation.` GitHub +3
2. 立刻做 A1 的 replication，而不是加新功能。
   你现在最需要的是验证“这个趋势是不是稳定出现”。直接按现有协议复跑三组模式，每组至少再做 2–3 次，保持同一任务、同一两张测试图、同一 `max_side=420` 、同一 shrink-only 口径。你的比较报告和 CoE prompt 都已经把协议冻结好了，所以现在最该做的是多次独立重跑，而不是再改 prompt 结构。 GitHub +1

我建议你这样编号：

```
run_010/011/012 = direct_answer replicate
run_013/014/015 = plain_guidance replicate
run_016/017/018 = coe_guided replicate
```

每次都新开一轮独立会话，把 `prompt_used.md` 、 `model_raw_response.md` 、 `src/` 、 `outputs/` 保留完整。这样你才能回答“CoE 的优势是偶然，还是重复出现”。

3. 把“指导效率”和“代码运行性能”拆成两张表。
   你现在的 `a1_guidance_eval_v0.csv` 很适合记录 `artifact_complete / runnable / correct / self_check / fix_rounds / time_to_first_working_min` ，而 codebook 也正是这么定义的；但 run 006/007/008 里其实还记录了另一类信息：生成代码本身在 `bing1.png` 和 `original.png` 上的 seam runtime，而且三者差异很大。也就是说，你当前把“提示方式是否更稳”和“生成代码跑得快不快”混在同一轮观察里了。下一步最好新开一张 `metrics/a1_codegen_perf_v0.csv` ，专门记 `image_name / method / width_runtime_s / height_runtime_s / total_runtime_s / output_ok` 。 GitHub +3

4. 把 failure cases 变成真正的 bug-repair benchmark。
   你已经有 `failure_cases/README.md` ，里面也明确说了要记录触发条件、错误类型、修复方式、修复轮次和是否回归通过。这个资产现在最值钱的用法，不是“展示我整理了 bug”，而是拿来做第二组研究实验：
   给三种模式分别喂 `bug_01/02/03` ，看谁能更快定位、修复并回归通过。 GitHub

这里我建议你新建：

`metrics/a1_failure_repair_eval_v0.csv`

字段用这几个就够：

`run_id,bug_id,mode,diagnosis_correct,patch_runnable,regression_pass,fix_rounds,time_to_fix_min,notes`

这一步其实会比“再比较一次从零写 seam carving”更能体现 CoE 的优势。因为你当前 CoE 的强点，不是在最终 `correct` 上拉开差距，而是在**检查覆盖、错误定位、修复可追踪性**上更强。这个判断和你现在的 CoE prompt设计、run 008 记录、比较报告是对得上的。 GitHub +2

5. 等 A1 的 replication 和 bug-repair 两条线都跑出结果，再启动 A2 的代码阶段。
   A2 的 task card 现在写得很清楚：先做 bootstrap，等 A1 的 run/metrics/report 管线对齐后再进入 A2 代码阶段。你现在已经完成了“对齐”的第一步，但还没完成“结果稳定化”的第二步。所以顺序应该是：
   A1 v0 复跑 → A1 错误修复基准 → A1 v1 小结 → A2 minimal implementation。 GitHub

你现在最适合给导师讲的一句话，我建议就用这个版本：

在 A1（Seam Carving）上，direct/plain/CoE 三种模式都能达到基本正确性；但 CoE 的主要增益目前不体现在最终正确率，而体现在首轮输出的结构化、自检覆盖和纠错可追踪性上。下一步将通过重复实验与错误修复基准，验证这一趋势是否稳定。 GitHub +1

你接下来优先做这 4 个文件就行：

- `report/a1_replication_plan_v0.md`
- `metrics/a1_codegen_perf_v0.csv`
- `metrics/a1_failure_repair_eval_v0.csv`
- 更新后的 `README.md`

做完这 4 个，我再继续带你把 run_010 到 run_018 的编号、填表规则和汇总报告一起定死。

来源

你这轮目标我理解为“把仓库从 A1 v0 首轮完成态推进到 Phase 2 入口”，我先快速核对当前 README 和现有 metrics 文件结构，然后直接落地一次首页状态更新并补齐你提到的新评估表模板。

我已经确认 README 确实落后于当前进度，接下来会直接改掉 Current status 与 Next milestones，并顺手把你提到的两张 Phase 2 指标表先建好空模板，方便立即开始复跑与修复基准。

现在开始执行文件修改：先更新首页叙事到“已完成 A1 v0 + 进入 Phase 2”，然后新增两张 CSV 模板以分离 guidance 效率与代码性能、并启动 bug-repair benchmark 的记录通道。

首页和两张新表已落地，我再快速复核一遍内容与路径，确保你可以直接按 run_010-018 开跑而不用再补结构。

已按你这条 Phase 2 思路完成仓库更新，且是可直接开跑的状态。

已完成修改

更新顶层 README 的 Current status 与 Next milestones，使其与当前真实进度一致。
见 README.md

新增“代码运行性能”独立指标表，避免与 guidance 质量混用。
见 a1_codegen_perf_v0.csv

新增“failure case 修复基准”指标表，支持 bug_01/02/03 的结构化对比。
见 a1_failure_repair_eval_v0.csv

本次落地后的状态与你给出的目标一致

首页已明确 A1 v0 完成、三模式跑通、failure cases 就绪、A2 仍 bootstrap-only。
下一阶段已改为 A1 replication -> A1 bug-repair benchmark -> A2 minimal implementation。

Quick links 已加入 A1 对比报告与两张新指标表，便于复跑时直接回填。
如果你要，我下一步可以直接把 run_010 到 run_018 的 run 记录模板批量建好（每个模板含固定协议检查项和回填提示），这样你就能按会话逐个执行并最小化记录成本。
目前的成果已经推到了github仓库：
https://github.com/YTYZ4/AI-for-research.git

Thought for 11m 47s ›

这一步是对的。我核对到三件关键事已经到位：A1 的 v0 比较结论已经写进 `a1_baseline_vs_coe_v0.md`，现阶段目标也明确是“先复跑验证趋势，再做 bug-repair benchmark”；同时你新建的两张 Phase 2 指标表 `a1_codegen_perf_v0.csv` 和 `a1_failure_repair_eval_v0.csv` 都已经落库。 GitHub +2

现在最该做的，确实就是你提到的那一步：**批量建 run_010 到 run_018 的模板。**
不过在建模板前，我建议再补一个很小的锁定动作：`a1_guidance_eval` 已经有 codebook，但你新建的两张表还没有字段口径说明；尤其 `a1_codegen_perf_v0.csv` 里的 `method` 现在是有歧义的，后面容易一会儿填“guidance 模式”，一会儿填“图像处理方法”。这类口径如果不先定，复跑一多就会乱。 GitHub +1

我建议你下一步按这个顺序来。

1. 先补两份 codebook。
   文件名直接定成：

- `metrics/a1_codegen_perf_codebook_v0.md`
- `metrics/a1_failure_repair_codebook_v0.md`

其中第一份把 `method` 明确成一个固定含义。为了和你现在的 Phase 2 目标对齐，我建议直接改成“guidance mode”，允许值只用：
`direct_answer / plain_guidance / coe_guided`。
如果你不想改 CSV 表头，那就在 codebook 里写死：`method = guidance mode`。
第二份则把 `diagnosis_correct / patch_runnable / regression_pass` 定成 0/1 口径，并把 `bug_id` 固定成 `bug_01_dp_boundary / bug_02_no_energy_recompute / bug_03_height_transpose`。 GitHub +1

2. 再批量建 run_010–018。
   编号直接固定，不要再犹豫：

- `run_010` `run_011` `run_012`：A1 `direct_answer` replication 1/2/3
- `run_013` `run_014` `run_015`：A1 `plain_guidance` replication 1/2/3
- `run_016` `run_017` `run_018`：A1 `coe_guided` replication 1/2/3

文件名模式统一成：

```
runs/<actual-date>_run_010_a1_direct_answer_rep1.md
runs/<actual-date>_run_011_a1_direct_answer_rep2.md
...
runs/<actual-date>_run_018_a1_coe_guided_rep3.md
```

日期用实际执行日期，编号不要变。

3. 每个 run 模板都用同一骨架。
   你直接复制这个结构就行：

```
# A1 replication run template

## Meta
- run_id:
- case_id: A1
- mode:
- replicate_id:
- prompt_file:
- task_card_version:
- protocol_file: report/a1_eval_protocol_v0.md
- env: mm26

## Fixed protocol checklist
- [ ] test images = bing1.png, original.png
- [ ] max_side = 420
- [ ] shrink only
- [ ] width shrink done
- [ ] height shrink done
- [ ] compare output generated

## Prompt and raw output
- prompt_used.md:
- model_raw_response.md:
- src path:
- outputs path:

## First-pass assessment
- artifact_complete:
- runnable:
- correct:
- self_check:
- first_error_type:
- first_error_summary:

## Fix log
- fix_rounds:
- fix_step_1:
```

```
- fix_step_2:
- final_working_time_min:

## Guidance metrics row to append
- CSV target: metrics/a1_guidance_eval_v0.csv
- row:

## Codegen performance rows to append
- CSV target: metrics/a1_codegen_perf_v0.csv
- row 1:
- row 2:

## Notes
- protocol deviation:
- notable behavior:
- comparison to prior runs:
```

这样你后面只需要填，不需要每次重新想结构。

4. 复跑时只允许变“会话”，不允许变“协议”。
   你现在的 `a1_baseline_vs_coe_v0.md` 已经把结论写得很谨慎：当前只是 v0，不该立刻扩大结论范围，而应该按同协议继续复跑，观察趋势是否稳定。你的 CoE prompt 也已经把 A1 CoE run 的角色结构和检查项锁死了。所以下一轮复跑最重要的纪律就是：**不要再改 prompt 结构，不要换图，不要改预处理，不要扩到 enlarge。**
   GitHub +1
5. 复跑完成后的最小汇总，也先提前定好。
   `run_018` 结束后，不要再拖，立刻写：

- `report/a1_replication_summary_v0.md`

只写这几项：

- 三种 mode 的 `runnable` 成功次数
- 三种 mode 的 `correct=2` 次数
- `fix_rounds` 的均值或中位数
- `time_to_first_working_min` 的均值或中位数
- `self_check=2` 的次数

这样你就会从“006/007/008 的单次比较”升级到“010–018 的重复实验趋势”。

还有一个小提醒：我这边看到 **raw README** 已经是你想要的 Phase 2 版本，但 GitHub 的 README 页面渲染我打开时还显示的是旧版状态和旧 milestones；这更像显示不同步，而不是仓库结构有问题。正式把仓库发给导师前，最好在浏览器里确认一下首页展示的是最新内容。 GitHub +1

下一步就别再扩目录了。先把这 11 个文件补出来：

- `metrics/a1_codegen_perf_codebook_v0.md`

- `metrics/a1_failure_repair_codebook_v0.md`
- `runs/run_010` 到 `runs/run_018` 的 9 个模板

建完之后，先实际跑 `run_010` 和 `run_016` 各一条，我再帮你看这轮 replication 的填表口径稳不稳。

来源